



Université Épiscopeale d'Haïti

Faculté des Sciences Informatiques

CAHIER DES CHARGES

Contacts

Application mobile de gestion de carnet d'adresses

FLUTTER

ANDROID

DART



COURS Développement mobile avec Flutter	OBJET Cahier des charges — application Contacts
PRÉPARÉ PAR Samuel Kensley Gaëtan	CODE ESTUDIANTIN 2450-C
E-MAIL samuelgaetan6969@gmail.com	NUMÉROS DE TÉLÉPHONE +509 3803 0260 • +509 4392 8285
PRÉSENTÉ AU PROFESSEUR Phawens Louis-Jean	DATE Juin 2026

Table des matières

	<i>Titre du projet</i>	1
1.	Présentation du projet	2
2.	Fonctionnalités principales	3
3.	Structure des données	5
4.	API utilisée	8
5.	Architecture du projet	9
6.	Répartition des tâches	10
	<i>Sources</i>	10

À PROPOS DE CE DOCUMENT

Ce cahier des charges décrit de manière complète et structurée l'application mobile **Contacts**, développée avec Flutter dans le cadre du cours de développement mobile — UNEPH, Juin 2026.



APPLICATION MOBILE • ANDROID

Contacts

Application mobile de gestion de carnet d'adresses

FLUTTER

ANDROID

DART

Application complète permettant de créer, modifier, organiser et partager des contacts personnels et professionnels depuis un smartphone Android, avec synchronisation distante via l'API randomuser.me.

1. Présentation du projet

1.1 Nom de l'application

Contacts — application native Android développée avec le framework Flutter (Dart).

1.2 Objectif principal

Fournir à l'utilisateur un gestionnaire de contacts personnel, moderne et autonome :

- Stocker localement un carnet d'adresses complet avec photos.
- Synchroniser des contacts fictifs depuis un service REST externe.
- Permettre les actions directes : appel, message, e-mail, partage.
- Offrir une expérience utilisateur soignée avec mode sombre, tri et groupement alphabétique.

1.3 Public cible

Toute personne possédant un smartphone Android (version 5.0 / API 21 ou supérieure) souhaitant disposer d'une application de contacts autonome, sans dépendance aux contacts système de l'appareil.

2. Fonctionnalités principales

2.1 Gestion des contacts (opérations CRUD)

Opération	Description
Création	Formulaire complet : photo, noms, téléphones, e-mails, adresses, dates, sites Web, relations, comptes de messagerie, sonnerie, notes.
Lecture	Affichage en liste groupée alphabétiquement avec barre de recherche en temps réel.
Modification	Formulaire pré-rempli avec toutes les données existantes.
Suppression douce	Déplacement vers la corbeille avec possibilité de restauration ultérieure.
Suppression définitive	Depuis la corbeille, après confirmation explicite de l'utilisateur.
Partage	Contenu du contact partagé en texte brut via le système Android (share_plus).

2.2 Écrans de l'application

L'application comporte au moins 5 écrans principaux décrits ci-après.

• Écran d'accueil › Titre de l'application et compteur de contacts. › Bouton « à » 3 points : Partager des contacts, Supprimer des contacts, Corbeille. › Barre de recherche en temps réel. › Liste alphabétique groupée (séparateurs A, B, C...). › État vide illustré lorsqu'aucun contact n'est présent. › Bouton FAB (+) pour ajouter un nouveau contact. › Navigation inférieure persistante (Contacts · Paramètres).
• Formulaire de création / modification › Sélection de photo depuis la galerie ou la caméra (gestion des permissions). › Emplacement de stockage : Téléphone / Autre. › Champs : Prénom·s, Nom de famille. › Numéros de téléphone : type (liste déroulante), champ formaté automatiquement, suppression, ajout multiple. › Adresses e-mail : même structure que les numéros. › Groupes. › Section extensible « Plus de champs » : adresses, dates importantes, sites Web, relations, comptes de messagerie, sonnerie, notes, titres prénominal et post-nominal. › Boutons Annuler / Enregistrer fixés en bas de l'écran.

- Fiche de contact
 - Barre supérieure : Retour, Partager, Modifier, Bouton « à » 3 points (Supprimer).
 - Avatar, nom complet, emplacement de stockage.
 - Boutons d'action : Appeler, Message, E-mail — lancent les applications système correspondantes.
 - Toutes les informations renseignées affichées par sections.
- Corbeille
 - État vide illustré si la corbeille est vide.
 - Liste des contacts supprimés avec date de suppression.
 - Actions par contact : Restaurer, Supprimer définitivement.
 - Actions en lot et option Vider la corbeille.
- Paramètres
 - Commutateur Mode sombre.
 - Tri des contacts : critère (prénom, nom, date de création, dernière modification) et ordre (ascendant / descendant).
 - Mon profil.
 - Importer des contacts depuis randomuser.me.
 - Stockage par défaut.
 - À propos : logo, version, développeur, institution.

2.3 Comportements automatiques

Comportement	Description
Groupement alphabétique	Contacts regroupés dynamiquement selon la première lettre du champ de tri sélectionné.
Formatage des numéros	Numéro automatiquement formaté à la saisie selon le préfixe pays détecté (+509, +1, +33...).
Infobulle (tooltip)	Un appui prolongé sur toute icône affiche son libellé.
SnackBar	Chaque action destructive est confirmée par un message contextuel avec option Annuler si pertinent.
Écran de chargement	Spinner affiché au démarrage pendant la lecture des données locales (SharedPreferences).
Écran d'erreur	Écran dédié si le chargement initial échoue, avec bouton Réessayer.

3. Structure des données

3.1 Modèle Contact — classe principale

Attribut	Type	Rôle
_id	String	Identifiant unique (UUID v4)
_firstName	String	Prénom·s (requis, validé)
_lastName	String	Nom de famille
_imagePath	String?	Chemin local ou URL distante
_storageLocation	StorageLocation	Emplacement (phone / other)
_phoneNumbers	List	Numéros de téléphone
_emails	List	Adresses e-mail
_addresses	List	Adresses postales
_importantDates	List	Dates importantes
_websites	List	Sites Web
_relations	List	Relations
_messagingAccounts	List	Comptes de messagerie
_isDeleted	bool	Contact en corbeille
_deletedAt	DateTime?	Date de mise en corbeille
_createdAt	DateTime	Date de création (immuable)
_updatedAt	DateTime	Dernière modification (auto)

Getters calculés

Getter	Type retourné	Description
<code>fullName</code>	<code>String</code>	Préfixe + prénom + nom + suffixe
<code>initials</code>	<code>String</code>	2 caractères max pour l'avatar
<code>primaryPhone</code>	<code>String</code>	Premier numéro de téléphone
<code>primaryEmail</code>	<code>String</code>	Première adresse e-mail

Méthodes principales

<code>addPhoneNumber / remove / update</code> Gestion des numéros de téléphone	<code>addEmail / removeEmail</code> Gestion des adresses e-mail
<code>addGroup / removeGroup</code> Gestion des groupes	<code>addAddress / removeAddress</code> Gestion des adresses
<code>softDelete()</code> Passe <code>_isDeleted</code> à true	<code>restore()</code> Passe <code>_isDeleted</code> à false
<code>toJson()</code> Séréalise en <code>Map<String, dynamic></code>	<code>Contact.fromJson(Map)</code> Déséréalise depuis <code>SharedPreferences</code>
<code>Contact.fromRandomUser(Map, id)</code> Déséréalise depuis <code>randomuser.me</code>	

3.2 Modèles secondaires

Classe	Attributs	Enum (types)
<code>PhoneNumber</code>	<code>_value</code> , <code>_type</code> , <code>_customLabel</code>	mobile, home, work, fax, other
<code>EmailAddress</code>	<code>_value</code> , <code>_type</code> , <code>_customLabel</code>	personal, work, other
<code>Address</code>	<code>_street</code> , <code>_city</code> , <code>_state</code> , <code>_postalCode</code> , <code>_country</code> , <code>_type</code>	home, work, other
<code>ImportantDate</code>	<code>_date</code> , <code>_type</code> , <code>_customLabel</code>	birthday, anniversary, other
<code>Website</code>	<code>_url</code> , <code>_type</code> , <code>_customLabel</code>	personal, work, blog, other
<code>Relation</code>	<code>_name</code> , <code>_type</code> , <code>_customLabel</code>	spouse, parent, child, sibling, friend, colleague, manager, other
<code>MessagingAccount</code>	<code>_handle</code> , <code>_platform</code> , <code>_customLabel</code>	whatsapp, facebook, telegram, instagram, twitter, snapchat, other

Chaque modèle secondaire implémente : constructeur principal avec attributs privés, `factory fromJson`, getters / setters, et `Map toJson()`.

3.3 Modèle de configuration — `SettingsProvider`

<code>_DARKMODE</code> <code>bool</code> — Thème sombre activé / désactivé	<code>_SORTCRITERIA</code> <code>SortCriteria</code> — <code>firstName</code> · <code>lastName</code> · <code>createdAt</code> · <code>updatedAt</code>
<code>_SORTORDER</code> <code>SortOrder</code> — <code>ascending</code> · <code>descending</code>	<code>_DEFAULTSTORAGE</code> <code>StorageLocation</code> — <code>phone</code> · <code>other</code>

4. API utilisée

4.1 Identification

NOM

Random User Generator

URL DE BASE

<https://randomuser.me/api/>

DOCUMENTATION

<https://randomuser.me/documentation>

AUTHENTIFICATION

Aucune — API publique et gratuite

4.2 Endpoint appelé

```
GET https://randomuser.me/api/?results=10&nat=fr

results = 10  → Nombre de contacts à générer
nat      = fr  → Nationalité (prénoms et noms français)
```

4.3 Format de la réponse JSON (extrait)

```
{
  "results": [
    {
      "name": { "title": "Mr", "first": "Jean-Paul", "last": "Dupont" },
      "email": "jean-paul.dupont@example.com",
      "phone": "03 45 67 89 01",
      "cell": "06 12 34 56 78",
      "picture": { "large": "https://randomuser.me/api/portraits/men/42.jpg" },
      "location": {
        "street": { "number": 12, "name": "Rue de la Paix" },
        "city": "Lyon",
        "state": "Auvergne-Rhône-Alpes",
        "postcode": "69001",
        "country": "France"
      }
    }
  ],
  "info": { "seed": "abc123", "results": 10, "page": 1, "version": "1.4" }
}
```

4.4 Correspondance JSON → `Contact.fromRandomUser`

Champ JSON	Attribut Contact
<code>name.first</code>	<code>_firstName</code>
<code>name.last</code>	<code>_lastName</code>
<code>email</code>	<code>_emails[0].value</code>
<code>phone</code>	<code>_phoneNumbers[0].value</code> — type : mobile
<code>cell</code>	<code>_phoneNumbers[1].value</code> — type : mobile
<code>picture.large</code>	<code>_imagePath</code> (URL réseau)
<code>location.*</code>	<code>_addresses[0]</code>

4.5 Gestion des erreurs réseau

Situation	Comportement
Timeout (> 15 s)	Exception propagée vers le provider
Code HTTP ≠ 200	Exception avec code et message
JSON invalide	Exception de parsing capturée
Affichage à l'UI	SnackBar rouge dans SettingsScreen

5. Architecture du projet

5.1 Arborescence des fichiers

```
contacts_app/
├─ android/app/src/main/
│   └─ AndroidManifest.xml      Permissions et configuration Android
├─ assets/images/
│   └─ ic_launcher.png         Icône de l'application (1024 × 1024 px)
└─ lib/
    ├─ main.dart                Point d'entrée · MultiProvider · écran de chargement
    ├─ models/
    │   ├─ contact.dart         Modèle principal (POO, fromJson/toJson)
    │   ├─ phone_number.dart    Numéro de téléphone avec type
    │   ├─ email_address.dart   Adresse e-mail avec type
    │   ├─ address.dart         Adresse postale
    │   ├─ important_date.dart  Dates importantes
    │   ├─ website.dart         Sites Web
    │   ├─ relation.dart        Relations
    │   └─ messaging_account.dart Comptes de messagerie
    ├─ providers/
    │   ├─ contacts_provider.dart État global (CRUD, recherche, tri)
    │   └─ settings_provider.dart Paramètres de l'application
    ├─ services/
    │   ├─ api_service.dart      GET randomuser.me · parsing JSON
    │   └─ storage_service.dart  Persistance locale (SharedPreferences)
    ├─ screens/                  Au moins 5 vues Flutter
    ├─ widgets/                  Composants réutilisables
    └─ utils/                    Thème · constantes · formatage · permissions
```

5.2 Flux de données

```
SharedPreferences ↔ StorageService ↔ ContactsProvider ↔ Screens
                        ↑
                    ApiService
                        ↑
                randomuser.me (GET)
```

5.3 Gestion d'état

- **ContactsProvider** — liste des contacts, opérations CRUD, recherche, importation API.
- **SettingsProvider** — préférences utilisateur : thème, tri, stockage par défaut.
- Les deux providers sont exposés via `MultiProvider` en racine de l'arbre de widgets pour un accès global.

5.4 Persistance locale

Toutes les données sont sérialisées en JSON et stockées via `SharedPreferences`. La clé `contacts_data` contient la liste complète des contacts actifs et ceux en corbeille.

5.5 Packages tiers utilisés

Package	Version	Usage
<code>provider</code>	<code>^6.1.2</code>	Gestion d'état
<code>http</code>	<code>^1.2.1</code>	Requêtes REST
<code>image_picker</code>	<code>^1.1.2</code>	Galerie / caméra
<code>permission_handler</code>	<code>^11.3.1</code>	Permissions Android
<code>shared_preferences</code>	<code>^2.3.2</code>	Persistance locale
<code>uuid</code>	<code>^4.4.2</code>	Génération d'ID uniques
<code>share_plus</code>	<code>^10.0.2</code>	Partage natif
<code>url_launcher</code>	<code>^6.3.0</code>	Appels, SMS, e-mails
<code>flutter_launcher_icons</code>	<code>^0.14.1</code>	Génération d'icône

6. Répartition des tâches

Ce projet est réalisé individuellement par Samuel Kensley Gaëtan (code 2450-C). Le tableau ci-dessous récapitule l'ensemble des tâches accomplies.

✓ Conception des modèles Dart (POO)	✓ Intégration de l'API randomuser.me
✓ Implémentation de image_picker	✓ Gestion des permissions Android
✓ Interface utilisateur (au moins 5 écrans)	✓ Persistance locale (SharedPreferences)
✓ Formatage automatique des numéros de téléphone	✓ Partage et actions directes (appel, SMS, e-mail)
✓ Mode sombre et paramètres utilisateur	✓ Corbeille avec restauration
✓ Rédaction du cahier des charges	

Sources

[1] **Flutter Documentation**
<https://docs.flutter.dev>

[3] **pub.dev — provider**
<https://pub.dev/packages/provider>

[5] **pub.dev — image_picker**
https://pub.dev/packages/image_picker

[7] **pub.dev — url_launcher**
https://pub.dev/packages/url_launcher

[9] **Random User Generator API**
<https://randomuser.me/documentation>

[11] **Android Developers — Permissions**
<https://developer.android.com/guide/topics/permissions/overview>

[13] **Figma — Design & Prototypage**
<https://www.figma.com>

[2] **Dart Language Tour**
<https://dart.dev/guides/language/language-tour>

[4] **pub.dev — http**
<https://pub.dev/packages/http>

[6] **pub.dev — permission_handler**
https://pub.dev/packages/permission_handler

[8] **pub.dev — share_plus**
https://pub.dev/packages/share_plus

[10] **Material Design 3**
<https://m3.material.io>

[12] **Flutter Cookbook — Networking**
<https://docs.flutter.dev/cookbook/networking/fdata>